

Computer Security Capstone

Project III: Ransomware Propagation and Payload

Chi-Yu Li (2026 Spring)

Computer Science Department

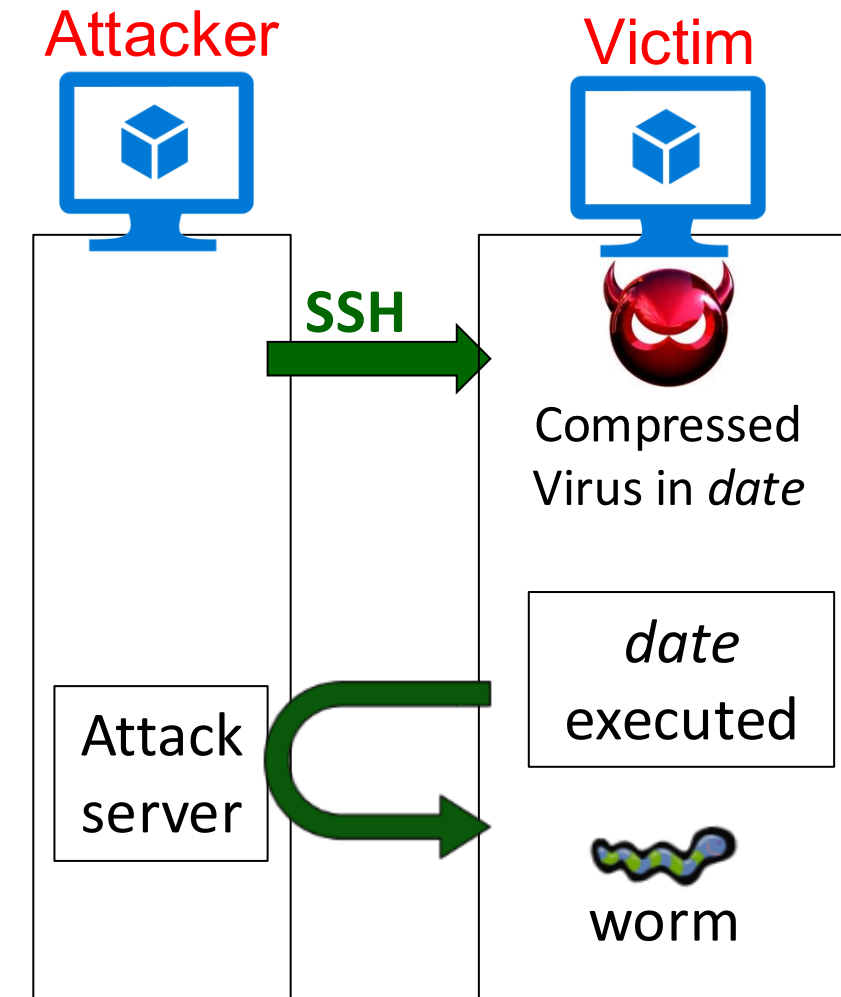
National Yang Ming Chiao Tung University

Goals

- Understand how a ransomware propagates and executes
- You will learn about the operation of
 - ❑ Dictionary attacks
 - ❑ Ciphering and deciphering
 - ❑ Digital signature
 - ❑ Compressed viruses
 - ❑ Worm propagation
 - ❑ Ransomware
 - ❑ PQC - KEM

Attack Scenario

- You are going to play the role of an attacker
- Assume that you know the IP of the victim and the username of his/her SSH account, you are asked to
 - ❑ Crack the victim's SSH password
 - ❑ Install a compressed virus in an affected program with attacker's signature generated by quantum safe algorithm
 - ❑ (Virus payload) Download and trigger a ransomware worm
- Consider the affected program: `/app/date` in victim.
 - ❑ When it is running, the virus payload is executed



Grading

- Task I: Crack SSH password (10%)
- Task II: Create a compression virus with the propagation of the ransomware worm (20%)
- Task III: Prepare the ransomware payload & PQC KEM (20%)
- Questions during demo (50%)

Task I: Crack SSH password

● Cracking the victim's password by launching a dictionary attack

- ❑ Assume that the victim's username is known as csc2026
- ❑ Assume that the password is created based on the victim's personal information
 - A file including the victim's personal information: `/app/victim.dat` in attacker container
 - One row contains an information entry
 - The password is composed of one or few information entries

● Hints

- ❑ Try strings combination in Python: **itertools**
- ❑ Automatic SSH and SFTP operation in Python: **paramiko**

Task II: Compression Virus with Ransomware Propagation

- Infect /app/date in victim by embedding your compression virus
- Infected 'date' shall
 - ❑ Keep the same size as the original 'date'
 - The original 'date' shall be compressed
 - ❑ Contain the virus payload and the functionality of the original 'date'
 - ❑ Finish the execution of the payload before the end of the 'date' execution
- The virus payload shall
 - ❑ Fetch a ransomware worm from the attack server
 - ❑ Execute the ransomware worm

Task II: Compression Virus with Ransomware Propagation

● Requirements

- ❑ The infection cannot leave any files except the infected 'date' on the victim container
- ❑ The last 512 bytes of the infected 'date' should be the signature generated by Dilithium3 algorithm
- ❑ The size of infected 'date' should be identical to the original one

● Hints

- ❑ Compress 'date' using a compression algorithm
- ❑ Minimize the virus size with various methods
 - e.g., using `/dev/tcp/host/port` to build tcp connections, gcc flags and strip
- ❑ Execute a program using the `exec()` family
- ❑ Use 'openssl dgst' to create & verify your signature

● Two major actions

- ## ● Requirements

- ## ● Hints

- [illegible]

Task III: Ransomware Payload & PQC KEM

● Interact with Kyber-512 Based Key Server

- ❑ The server owns the private key, and you acts as a client.
- ❑ You receive a public key from the server, uses it to create a ciphertext and a shared secret using Kyber-512, then sends only the ciphertext back to server.
- ❑ The server decapsulates that ciphertext and computes the shared secret. If the shared is the same , the server would return a pair of AES Key and IV for your jpg image encryption.

● Note

- ❑ You should not hardcode the Key and IV, fetch it dynamically when you run the ransomware.

Task III: Ransomware Payload & PQC KEM

● Server API Endpoint

□ GET <http://140.113.207.249:8080/kem/public-key>

■ Example response

- {"session_id": "example-session-id", "public_key": "base64-public-key"}

□ POST <http://140.113.207.249:8080/kem/decapsulate>

■ Content-Type: application/json

■ Request body:

- {"session_id": "example-session-id", "ciphertext": "base64-ciphertext"}

■ Example response :

- { "key": "32_hex_characters", "iv": "16_hex_characters" }
- Use this Key and IV to encrypt your jpg files.

Task III: Ransomware Payload & PQC KEM

- Note : You **must log the Key and IV** you obtained from the key server in the terminal of the attacker container.
 - ❑ Other debug logs are allowed.
- If the log cannot be found in the terminal, we would regard image decryption as failed.
- Log Format :

```
root@43964616d46e:/home/csc2026/314551003# ./crack_attack 172.18.0.3 172.18.0.2 8888
[*] KEM exchange with http://140.113.207.249:8080 ...
[*] key=515c11d8efecbbc9139067db3cef5421 iv=5d1348908979e77a
[*] Starting password cracking...
```

Requirements

- You need to develop/run your program in the given containers.
 - ▣ Resource is provided in /app
 - username/password: csc2026/csc2026
 - Note: the victim's password will be changed based on a new victim.dat file in the demo
 - ▣ Please complete your project under the path: /home/csc2026/\${yourstudentID}
- You are allowed to use C/C++, Shell Script or/and Python (Recommended).
- You are allowed to team up; each team has at most 2 students
 - ▣ Teams: discussions are allowed, but no collaboration
- Please submit your source codes to E3
- Please email your questions to csc2026@nemslab.tw

Important: How to set up environment?

- Build the project3 image
 - ▣ Linux: “ docker compose build ”
- Create the project3 containers
 - ▣ Linux: “ docker compose up -d ”
- Enter the attacker container
 - ▣ Linux: “ docker exec -it attacker bash ”
- Enter the victim container
 - ▣ Linux: “ docker exec -it victim bash ”

Important: How to Prepare Your Attack Programs?

- Must provide a **Makefile** which compiles your source codes into at least two executable files: **crack_attack** and **attack_server**
- Test requirements for your program
 - ❑ Must be run in the given containers without any additional tools or libraries
 - ❑ Must work for the following two test commands
 - `./crack_attack <Victim IP> <Attacker IP> <Attacker port>`
 - `./attack_server <Attacker port>`

Important: Major Demo Steps (Not Exactly the Same)

● Attacker container

- ❑ Run “make” to compile your source codes
- ❑ Run “./attacker_server <Attacker port>” to set up the attacker server
- ❑ Run “./crack_attack <Victim IP> <Attacker IP> <Attacker port>” to crack the victim’s password and infect ‘date’ in victim

● Victim container

- ❑ Check the size of ‘date’ and any additional files generated
- ❑ Run 1 commands of ‘date’
 - ‘date’ shall perform its original function
 - Only the jpg files in /app/Pictures are encrypted with the given security context
 - A ransom graph shall show up
- ❑ Check whether the encrypted files can be decrypted

Project Submission

- Due date: 5/20 11:55 PM (Late submissions will not be accepted)
- Makeup submission (75 % at most): After the final
- Submission rules
 - ❑ Put all your files into a directory and name it using your student ID(s)
 - If your team has two members, please concatenate your IDs separated by “-”
 - ❑ Zip the directory and upload the zip file to E3
 - ❑ Wrong folder format or containing __MACOSX would receive 10 points deduction.
 - ❑ A sample of the zip file: 1234567-7654321.zip
 - 1234567-7654321
 - | Makefile
 - | crack_attack.c
 - | attack_server.c
 - | {other files you have}

Appendix: Quantum-Safe Cryptos (QSC)

- Conventional cryptographic algorithm bases on some mathematic problems that is difficult to be cracked via modern computers
- Quantum computers can use Shor's algorithm to solve these kinds of problems efficiently, thus threatens several crypto algorithms (e.g., RSA, ECDSA, ECDHE, ...)
- New algorithms that can resist this kind of threats are called quantum-safe cryptos

Appendix: QSC in Practice

- OpenSSL introduces provider mechanism to embrace algorithms from third-parties
- Open Quantum Safe Project implements algorithms and adapt them with the provider interface into OpenSSL framework
- Check [oqs-provider / USAGE.md](#) to know its usage
- You can check the availability of QSC in your container with the command:

```
$ openssl list -providers
Providers:
oqsprovider
  name: OpenSSL OQS Provider
  status: active
```

Appendix: The Process of PQC KEM (Kyber-512)

1. Key generation (server side):

- ❑ The server generates a public/private key pair using the Kyber-512 parameters (based on lattice cryptography, specifically the Module-LWE problem). It sends the public key to the client while keeping the private key secret.

2. Encapsulation (client side):

- ❑ The client uses the server's public key to create a shared secret and a corresponding ciphertext. It sends this ciphertext to the server; the shared secret is kept locally by the client.

3. Decapsulation (server side):

- ❑ The server uses its private key to decrypt (decapsulate) the ciphertext and recover the same shared secret. Now both client and server independently hold the identical secret, which can be used for symmetric encryption (e.g., secure communication).

Questions?